SONGAI: Sistema Inteligente de Clasificación Vocal Mediante Redes Neuronales Artificiales Multicapa

Resumen

El presente proyecto describe el desarrollo de SONGAI, un sistema inteligente orientado al análisis y clasificación de voz mediante técnicas de procesamiento digital de audio y redes neuronales artificiales multicapa. El sistema fue diseñado utilizando una arquitectura cliente-servidor con React para el frontend y Flask para el backend, integrando bibliotecas de inteligencia artificial y procesamiento de audio como TensorFlow, Librosa y Scikit-Learn.

El objetivo principal fue desarrollar una red neuronal capaz de identificar características vocales a partir de grabaciones de audio y clasificarlas en categorías vocales generales, específicamente voces agudas y graves. Para ello se utilizó el dataset VocalSet, reconocido en investigación de audio y machine learning aplicado a voz cantada.

Durante el desarrollo se implementó una metodología basada en extracción de características acústicas, normalización de datos, entrenamiento supervisado mediante una red neuronal multicapa (MLP) y posterior integración del modelo en una aplicación web interactiva. Los resultados obtenidos muestran que el sistema logró clasificaciones exitosas en tiempo real, alcanzando altos niveles de confianza en las predicciones.

Palabras clave— Redes neuronales artificiales, Deep Learning, Imagen, Clasificación vocal, TensorFlow, Procesamiento de audio, Flask, React.

I. INTRODUCCIÓN

El crecimiento de las técnicas de inteligencia artificial ha permitido el desarrollo de sistemas capaces de interpretar señales complejas como imágenes, texto y audio. Dentro del procesamiento de audio, una de las áreas con mayor crecimiento es la clasificación automática de voz mediante redes neuronales.

La identificación vocal tiene aplicaciones en múltiples áreas, incluyendo asistentes virtuales, sistemas biométricos, análisis musical, recomendación de canciones y entrenamiento vocal. Gracias al uso de redes neuronales profundas, actualmente es posible detectar patrones acústicos complejos imposibles de analizar mediante métodos tradicionales.

El presente trabajo propone el desarrollo de SONGAI, una aplicación web inteligente capaz de capturar audio desde el navegador, extraer características relevantes de la señal de voz y clasificar el tipo vocal del usuario utilizando una red neuronal multicapa.

El sistema fue desarrollado como una implementación práctica de conceptos vistos en la asignatura de Redes Neuronales Artificiales y Aprendizaje Profundo, integrando múltiples tecnologías modernas de desarrollo e inteligencia artificial.

II. OBJETIVO GENERAL

Desarrollar un sistema inteligente basado en redes neuronales artificiales multicapa capaz de clasificar características vocales a partir de grabaciones de audio en tiempo real.

Objetivos específicos

- Implementar un sistema de captura de audio mediante navegador web.
- Extraer características acústicas relevantes de señales de voz.
- Entrenar una red neuronal multicapa utilizando TensorFlow.
- Integrar el modelo entrenado en una aplicación web funcional.
- Mostrar resultados y probabilidades de clasificación en tiempo real.

III. MARCO TEÓRICO

A. Redes Neuronales Artificiales

Las redes neuronales artificiales son modelos computacionales inspirados en el funcionamiento del cerebro humano. Están compuestas por neuronas artificiales conectadas mediante pesos sinápticos que permiten aprender patrones a partir de datos.

En este proyecto se utilizó una red neuronal multicapa (MLP), la cual consiste en:

- Capa de entrada
- Capas ocultas
- Capa de salida

Las capas ocultas permiten detectar patrones complejos dentro de los datos de entrada mediante funciones de activación no lineales.

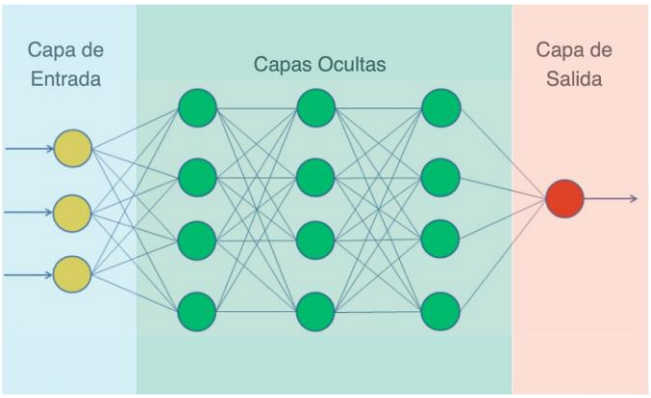


Imagen referencial 1.

B. Deep Learning

El aprendizaje profundo o Deep Learning es una rama del machine learning basada en redes neuronales profundas con múltiples capas. Estas redes permiten modelar relaciones complejas y son ampliamente utilizadas en reconocimiento de voz, visión computacional y procesamiento de lenguaje natural.

C. Procesamiento Digital de Audio

El procesamiento de audio permite transformar señales sonoras en representaciones numéricas analizables por algoritmos de inteligencia artificial.

En este proyecto se utilizaron las siguientes características acústicas:

- Pitch
- MFCC (Mel Frequency Cepstral Coefficients)
- Zero Crossing Rate
- RMS Energy
- Spectral Centroid

Estas características permiten representar propiedades relevantes del timbre y comportamiento de la voz.

IV. METODOLOGÍA

A. Arquitectura General del Sistema

El sistema SONGAI fue dividido en dos componentes principales:

Frontend

Desarrollado en React.js.

Funciones principales:

- Captura de audio desde micrófono
- Comunicación con el backend mediante API REST
- Visualización de resultados
- Interfaz interactiva

Backend

Desarrollado en Flask.

Funciones principales:

- Recepción de archivos de audio

- Extracción de características
- Predicción mediante IA
- Envío de resultados al frontend

B. Dataset Utilizado

Se utilizó el dataset VocalSet, un conjunto de datos ampliamente utilizado en investigación de voz cantada.

Características del dataset:

- Grabaciones de múltiples cantantes
- Archivos WAV
- Diferentes técnicas vocales
- Voces masculinas y femeninas

Para simplificar el entrenamiento inicial se trabajó únicamente con la categoría:

- long_tones

Además, las clases fueron agrupadas inicialmente en:

- Aguda
- Grave

C. Extracción de Características

El procesamiento de audio fue realizado utilizando la biblioteca Librosa.

Las características extraídas fueron:

Pitch

Representa la frecuencia fundamental predominante de la voz.

MFCC

Los coeficientes MFCC representan características espectrales utilizadas ampliamente en reconocimiento de voz.

Zero Crossing Rate

Indica la cantidad de veces que la señal cambia de signo.

RMS Energy

Representa la energía promedio de la señal de audio.

Spectral Centroid

Representa el centro de masa del espectro de frecuencias.

D. Preparación del Dataset

Se desarrolló un script llamado prepare_dataset.py encargado de:

1. Recorrer automáticamente las carpetas del dataset.
2. Leer archivos WAV.
3. Extraer características acústicas.
4. Generar un dataset tabular.
5. Exportar la información a un archivo CSV.

El resultado final fue un archivo llamado:

voice_dataset.csv

pitch	zcr	rms	spectral_cent	mfcc_1	mfcc_2	m
316.51028	0.08053639	0.06298634	2057.54048	-410.29703	18.154783	
316.0674	0.08187361	0.07239254	2071.01167	-411.2713	2.0941348	
315.2451	0.07631673	0.07131769	2145.41	-427.90237	-3.6660125	
331.18384	0.06289022	0.05018035	1950.19341	-453.73843	17.173077	
328.2622	0.05826721	0.06363653	1831.4725	-446.0929	13.853024	
343.16986	0.06955517	0.00563942	2043.30731	-596.88245	53.725014	
319.88327	0.05795028	0.00331478	1954.31979	-640.684	51.517124	
306.0461	0.05576888	0.00404663	2005.79465	650.89154	53.02474	

Imagen referencial 2.

Este archivo fue utilizado posteriormente para entrenar la red neuronal.

E. Entrenamiento de la Red Neuronal

El entrenamiento fue desarrollado utilizando TensorFlow y Keras.

Arquitectura utilizada

- Capa de entrada
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(32, ReLU)
- Dense(2, Softmax)

Función de pérdida

Categorical Crossentropy.

Optimizador

Adam.

Configuración de entrenamiento

- Epochs: 30
- Batch Size: 32
- División entrenamiento/prueba: 80/20

F. Integración del Modelo

Una vez entrenado el modelo, se almacenaron los siguientes archivos:

- voice_model.h5
- scaler.pkl
- label_encoder.pkl

Posteriormente se desarrolló un sistema de predicción en tiempo real utilizando Flask.

El flujo general del sistema es el siguiente:

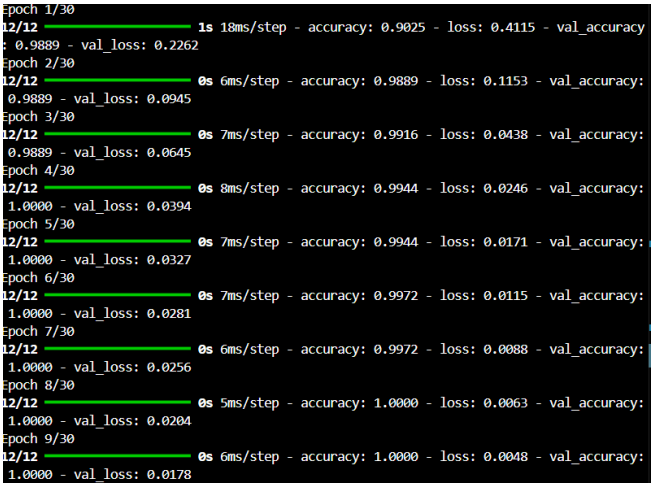
Captura de audio → Extracción de características → Normalización → Predicción IA → Visualización de resultados

V. RESULTADOS

A. Entrenamiento del Modelo

Durante el entrenamiento se observó una mejora progresiva en la precisión del modelo conforme avanzaban las épocas.

El sistema logró clasificar correctamente muestras vocales en categorías agudas y graves utilizando características acústicas extraídas automáticamente.



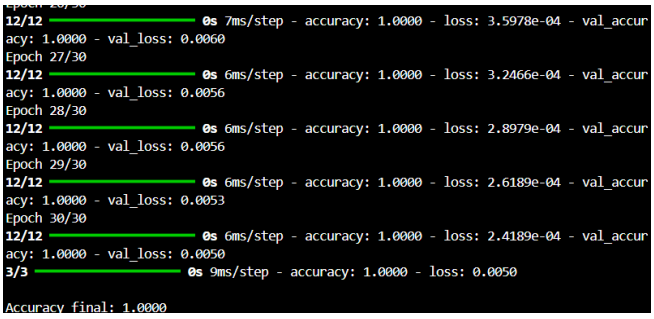


Imagen de captura de epochs y accuracy del entrenamiento.

B. Resultados en Tiempo Real

La aplicación logró capturar audio desde el navegador y procesarlo correctamente mediante el backend en Flask.

El modelo realizó predicciones exitosas mostrando:

- Tipo de voz detectado
- Nivel de confianza
- Probabilidades de clasificación

Ejemplo de resultado

Tipo de voz: Grave

Confianza IA: 99.98%

Probabilidades:

- Aguda: 0.02%
- Grave: 99.98%





Imagen de captura de resultados Frontend.

C. Análisis de Resultados

Los resultados obtenidos indican que la red neuronal fue capaz de aprender patrones relevantes dentro de las señales de audio.

La utilización de múltiples características acústicas permitió mejorar la capacidad de clasificación comparado con sistemas basados únicamente en pitch.

Asimismo, la arquitectura multicapa permitió detectar relaciones complejas entre los datos de entrada.

VI. PROBLEMAS ENCONTRADOS

Durante el desarrollo se presentaron distintos problemas técnicos, entre ellos:

- Compatibilidad entre TensorFlow y Python 3.13.
- Problemas de lectura de archivos WEBM.
- Configuración de FFmpeg.
- Errores de integración entre frontend y backend.
- Problemas de normalización y codificación de etiquetas.

Cada problema fue solucionado mediante ajustes en librerías, formatos de audio y estructura del pipeline de IA.

VII. CONCLUSIONES

El proyecto SONGAI permitió implementar exitosamente un sistema de clasificación vocal utilizando redes neuronales artificiales multicapa.

Se logró integrar procesamiento digital de audio, extracción de características acústicas y deep learning dentro de una arquitectura web funcional.

Los resultados obtenidos demostraron que el modelo es capaz de identificar diferencias vocales y generar predicciones con altos niveles de confianza.

Además, el proyecto permitió aplicar de manera práctica conocimientos de:

- Redes neuronales
- Deep Learning
- Procesamiento digital de señales
- Machine Learning
- Desarrollo web
- APIs REST

Como trabajo futuro se propone:

- Incorporar más clases vocales.
- Utilizar redes convolucionales (CNN).
- Implementar espectrogramas.
- Mejorar precisión mediante mayor cantidad de datos.
- Añadir recomendación automática de tonalidades musicales.

VIII. REFERENCIAS

[1] F. Wilkins, “VocalSet: A Singing Voice Dataset,” Zenodo, 2018.

[2] TensorFlow Documentation. Disponible en: https://www.tensorflow.org/

[3] Librosa Documentation. Disponible en: https://librosa.org/

[4] Scikit-Learn Documentation. Disponible en: https://scikit-learn.org/

[5] Flask Documentation. Disponible en: https://flask.palletsprojects.com/

[6] React Documentation. Disponible en: https://react.dev/